

Customer: "I don't use DOS. What would happen if I deleted that directory?"

- Customer Service Records

CSE 312

Operating Systems

Spring 2026

Syllabus

© 2013-2026 Yakup Genç

1

Today

- Processes and threads

2

Multiprogramming OSs

- Multiplexing increases complexity
 - Car Analogy (by Rosenblum):
 - Dedicated road per car would be incredibly inefficient, so cars share freeway. Must manage this.
 - (abstraction) different lanes per direction
 - (synchronization) traffic lights
 - (increase capacity) build more roads
- More utilization creates contention
 - (decrease demand) slow down
 - (backoff/retry) use highway during off-peak hours
 - (refuse service, quotas) force people into public transportation
 - (system collapse) traffic jams

Allocation of Resources

- OS must decide who gets to use what resource
- Three approaches
 - The admin (boss) can specify
 - The user can specify
 - What if user lies?
 - What if user doesn't know?
 - Figure it out through feedback
 - Problem: how to tell power users from resource hogs?

Goals for Resource Management

- Fairness
 - Assign resources equitably
- Differential Responsiveness
 - Cater to individual applications' needs
- Efficiency
 - Maximize throughput, minimize response time, support as many apps as you can
- These goals are often conflicting
 - All about trade-offs

Single program vs Multiprogramming

- Computers don't really run multiple programs simultaneously; it just appears that way
- Each process runs to completion, but intermixed with other processes

Time	1	2	3	4	5	6	7	8	9		
Process 1	-		-		-		-				
Process 2			-		-						
Process 3				-		-		-			
CPU Usage	1	2	1	3	2	1	3	1	3		

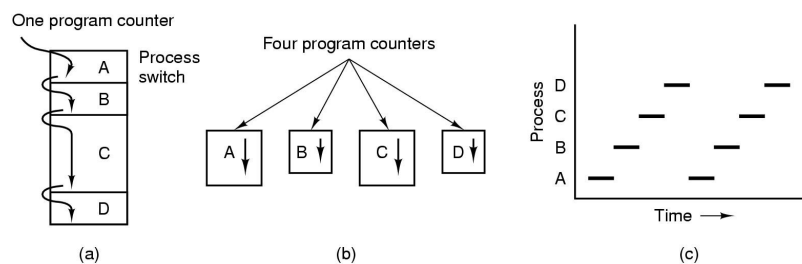
- The exact timing pattern varies for each process
- Note the idle times

Process Time

- What matters is that each process eventually finishes
- You start reading a book, put it down for a while, pick it back up and resume where you left off
- As long as you finish soon enough — whatever that means — the exact time doesn't matter

7

The Process Model



- (a) Multiprogramming of four programs
 (b) Conceptual model of four independent, sequential processes
 (c) Only one program is active at once

8

Real-Time Scheduling

- In some environments, processes need to run at the right time
- Think of process control computers — valves must open and close promptly
- Not suitable for this paradigm

What's a Process?

- No rigorous definition!
- Precise characteristics differ on different systems
- Common themes: separately scheduled; some measure of isolation

Separately Scheduled

- On all systems, processes can compete for the CPU
- One process blocking or being pre-empted lets another process run
- On some systems, a process can be composed of several threads that themselves can compete for the CPU
- On multiprocessor (multi-CPU) machines, different processes can execute truly simultaneously on different CPUs

Isolation

- Termination protection
 - OS must protect/isolate applications from each other, and OS from applications
 - This requirement is absolute
 - In Pintos: if one application crashes, kernel should not!
- Address space
- Security context
- Other system-related state

Isolation

- Three techniques
 - Preemption
 - Interposition
 - Privilege

Isolation – Preemption

- Access to resources can be granted and revoked
 - CPU, Memory, Printer, “abstract” resources: files, sockets
- Example: CPU Preemption using interrupts
 - Hardware timer interrupt invokes OS
 - OS checks if current program should be preempted, done every 1ms in Linux
 - Solves infinite loop problem!
- Q: Does this technique work with all resources equally?

Isolation – Interposition

- OS hides the hardware
- Application have to go through OS to access resources
- OS can interpose checks:
 - Validity (Address Translation)
 - Permission (Security Policy)
 - Resource Constraints (Quotas)

Isolation – Privilege

- Two fundamental modes:
 - “kernel mode” –privileged
 - aka system, supervisor or monitor mode
 - Intel calls its PL0, Privilege Level 0 on x86
 - “user mode” –non-privileged
 - PL3 on x86
- Bit in CPU – controls operation of CPU
 - Protection operations can only be performed in kernel mode Example: hlt
 - Carefully control transitions between user & kernel mode

Termination

- Processes are generally isolated from failures of other processes
- Termination of a process — normal or abnormal — does affect other processes
- Often a reason for process creation: let failures happen in an isolated setting, with minimal cleanup needed

Address Space

- Processes often have separate address spaces from each other
- Changes to memory in one process do not affect other processes
- May or may not use virtual memory to provide overlapped address space — on early PDP-11 Unix systems, all processes started at location 0

Exceptions. . .

- On OS/360, the job was the unit of memory protection; all “tasks” (the OS word for “process”) shared memory
- On some versions of MVS, all jobs have parts of kernel memory available at the same addresses
- On Unix systems, program instructions — but not data — are shared among different processes; this often includes shared libraries
- Unix processes can arrange to share certain memory areas
- Files can be mapped to memory areas; on different processes, the same data can thus appear at different addresses

Security Context

- Access credentials — UID on Unix — are process-specific
- SetUID applies to a process
- On some systems, process can share credentials

System State

- Open files
- Current working directory
- Trap-handling state
- Permissions for newly-created files
- Often much more

A Historical Note

- PDP-11s had a 16-bit address space: 65536 bytes
- The page size — the granularity of memory protection — was 4096 bytes
- Even with tiny programs, that meant at most 16 processes if they shared address space
- Separate address space per process was a necessity

Processes and System Calls

- Suppose a process issues a system call. What happens in the kernel?
- Interrupt hardware saves old PSW; loads new PSW
- Software interrupt handler saves registers; branches to system call dispatcher
- Dispatcher figures out which system call it is, and calls that subroutine
- That subroutine may call others
- We need a stack

The Kernel Stack

- As discussed previously, cannot trust user-level setup
- Must have a kernel stack
- Stack size is limited — watch for too-deep recursion!
- Where is this stack?

Per-Process Stacks

- Suppose this system call blocks waiting for I/O
- Another process can run; what if it issues a system call?
- It can't share the first process' stack, because that one may need to be used while this one is active
- We need a separate kernel stack per process!

Per-Process Stacks

- Actually, we need a lot of state per process
- The basic per-process structure on Linux (task struct) is 175 lines of C, and it points to other per-process structures
- What's in them?
- Two broad classes: fields needed when running and fields needed when deciding whether or not to run the process

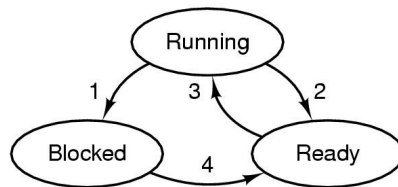
Per-Process State: Always Needed

- Process state: running, ready, blocked
- What it's blocked on
- Resource (CPU, RAM, I/O, etc.) usage history
- Priority
- Signal status
- ProcessID, process group
- Pointers to other fields
- Memory allocations

Per-Process State: Needed When Running

- Kernel stack
- PSW, program counter
- Open file descriptors
- Some state information, such as current directory
- User credentials

Process States



1. Process blocks for input
2. Scheduler picks another process
3. Scheduler picks this process
4. Input becomes available

Note that when making the transition from Blocked to Ready, it may be necessary to copy in some state data from disk. This, of course, is itself a blocking operation.

Process Creation

- Events which cause process creation:
 - System initialization
 - Execution of a process creation system call by a running process
 - A user request to create a new process
 - Initiation of a batch job
- The first process is created at boot time
- Any process, including that one, can create new processes
- Details differ widely between operating systems

Process Creation on Unix

- Basic operation: **fork()**
- Creates an exact copy of the process, code, data, and state
- Only difference: parent is passed back processID of child; child is passed back 0
- The child process typically manipulates some file descriptors, then **exec()**s some other program
- Note: virtually all Unix commands create separate processes.

Process Creation on Windows

- The **CreateProcess** call creates processes on Windows
- Executing a new program is part of the process creation mechanism
- 10 parameters control the program to be executed, window creation, priority, security attributes, file inheritance, and much more
- The Windows call does more for you, but is it simpler?

Process Relationships in Unix

- A newly-created process is a child of the parent process
- When a child process terminates, its resource consumption is passed up to the parent
- The parent process is notified when a child terminates, and needs to “reap” it (via the `wait()` system call)
- If not, the process remains a zombie
- Processes whose parent dies become children of process 1

Process Groups

- Related processes — say, all the elements of a pipeline — form a process group
- Certain signals — interrupts to a process — are sent to all members of a process group
- Thus, if you hit `^C`, all of the processes are killed

Windows Process Relationships

- All Windows processes are siblings; there are no other relationships
- When a process creates another process, it receives a process handle that can be used to control that process
- The process handle can be passed around to other processes

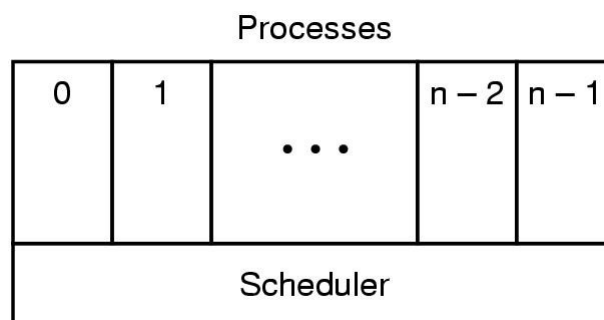
Process Termination

- Events which cause process termination:
 - Normal exit (voluntary)
 - Error exit (voluntary)
 - Fatal error (involuntary)
 - Killed by another process (involuntary)
- When a process terminates, its resources must be freed
- Some of these resources including open files; closing a file can block
- Termination isn't easy, and may not terminate quickly. . .

Creating a Process — Overview

- Parent issues a system call
- Interrupt handler invokes the kernel
- Kernel creates the process
- At some point, it runs

Implementation of Processes



The lowest layer of a process-structured operating system handles interrupts and scheduling. Above that layer are sequential processes.

Implementation of Processes

Process management	Memory management	File management
Registers	Pointer to text segment info	Root directory
Program counter	Pointer to data segment info	Working directory
Program status word	Pointer to stack segment info	File descriptors
Stack pointer		User ID
Process state		Group ID
Priority		
Scheduling parameters		
Process ID		
Parent process		
Process group		
Signals		
Time when process started		
CPU time used		
Children's CPU time		
Time of next alarm		

Some of the fields of a typical process table entry.

Issuing a System Call

- **Parent** Issues (machine-language) system call instruction
- **Hardware** Old PSW and program counter are saved
- **Hardware** New PSW and program counter are loaded
- **Assembler** Registers are saved in current process' kernel data structure
- **Assembler** System call dispatcher is invoked
- **C** Process creation routine invoked

PSW

Note :

The value presented by RS0 and RS1 selects the corresponding register bank.

PSW : Program Status Word (Bit Addressable)

CY	AC	F0	RS1	RS0	OV	F1	P
----	----	----	-----	-----	----	----	---

CY	PSW.7	Carry Flag.
AC	PSW.6	Auxiliary Carry Flag.
F0	PSW.5	Flag 0 available to the user for general purpose.
RS1	PSW.4	Register Bank selector bit 1 (SEE NOTE).
RS0	PSW.3	Register Bank selector bit 0 (SEE NOTE).
OV	PSW.2	Overflow Flag.
F1	PSW.1	Flag F1 available to the user for general purpose.
P	PSW.0	Parity flag. Set/cleared by hardware each instruction cycle to indicate an odd/even number of "1" bits in the accumulator.

RS1	RS0	REGISTER BANK	ADDRESS
0	0	0	00H-07H
0	1	1	08H-0FH
1	0	2	10H-17H
1	1	3	18H-1FH

Implementation of Processes

1. Hardware stacks program counter, etc.
2. Hardware loads new program counter from interrupt vector.
3. Assembly language procedure saves registers.
4. Assembly language procedure sets up new stack.
5. C interrupt service runs (typically reads and buffers input).
6. Scheduler decides which process is to run next.
7. C procedure returns to the assembly code.
8. Assembly language procedure starts up new current process.

Skeleton of what the lowest level of the operating system does when an interrupt occurs.

Process Creation Routine

- Verify that resources are available
 - Process table entry
 - User's process quota
- Create new process table entry
- Copy inherited data to new process
- Make sure "saved" registers are correct, including return value to indicate it's a child process
- Return to system call dispatcher

Returning From the Kernel

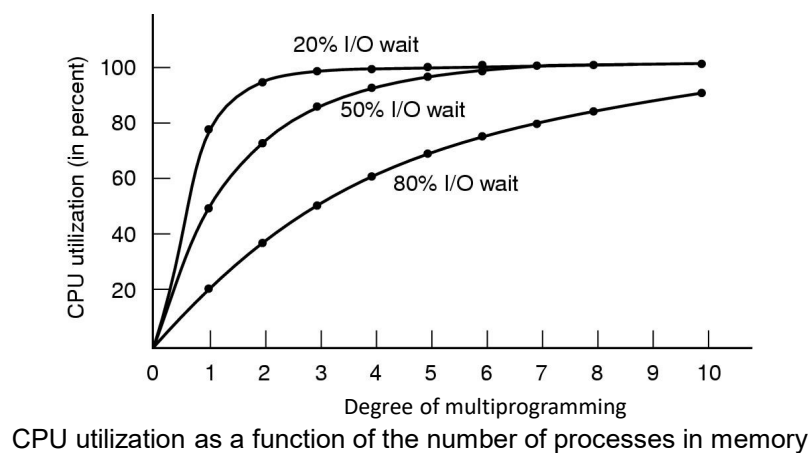
- When the new process is created, the dispatcher invokes the scheduler
- The scheduler decides which process will run next — the parent, the child, or some other process entirely
- Assembler code restores registers for whatever process is the next to run
- The old PSW and program counter are reloaded by "return from interrupt" instruction

Notes...

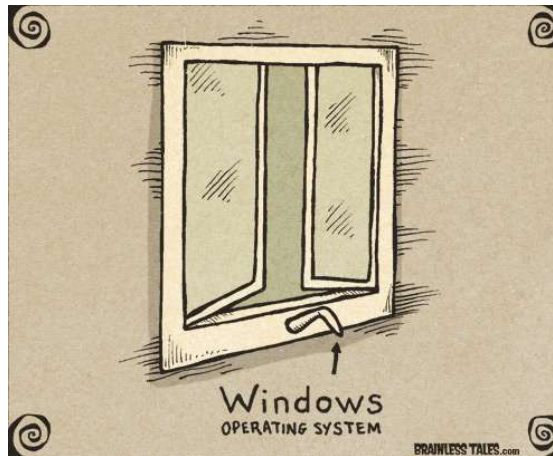
- How the new process behaves is completely determined by what is put into the process structure
- “Registers” aren’t a C concept, but the contents of the any process’ registers are determined by what is put into this structure

45

Modeling Multiprogramming



46



Spring 2026

Operating Systems

47

47

What is a Thread?

- Similar to a process
- Separate "thread of execution"
- Each thread has a separate stack, program counter, and run state

Spring 2026

Operating Systems

48

48

Differences Between Processes and Threads

- Threads share the same address space
- Threads always have the same security context (i.e., UID)
- Threads share file state
 - A child process will inherit open files from the parent, but won't see newly-opened files. A child thread will see them.
- Non-preemptive scheduling

Non-preemptive Scheduling

- There is no timer to make a thread yield the CPU
- Threads must voluntarily yield control to let another thread run
- Thread history isn't taken into account by the scheduler
- Threads are co-operative, not competitive

Differences Between Processes and Threads

- Process – These are all possible definitions:
 - A program in execution
 - An instance of a program running on a computer
 - Schedulable entity (*)
 - Unit of resource ownership
 - Unit of protection
 - Execution sequence (*) + current state (*) + set of resources

(*) can be said of threads as well

Alternative Definition

- Thread:
 - Execution sequence + CPU state (registers + stack)
- Process:
 - n Threads + Resources shared by them (specifically: accessible heap memory, global variables, file descriptors, etc.)
- In most contemporary OS, $n \geq 1$
- In Pintos, $n=1$: a process is a thread – as in traditional Unix

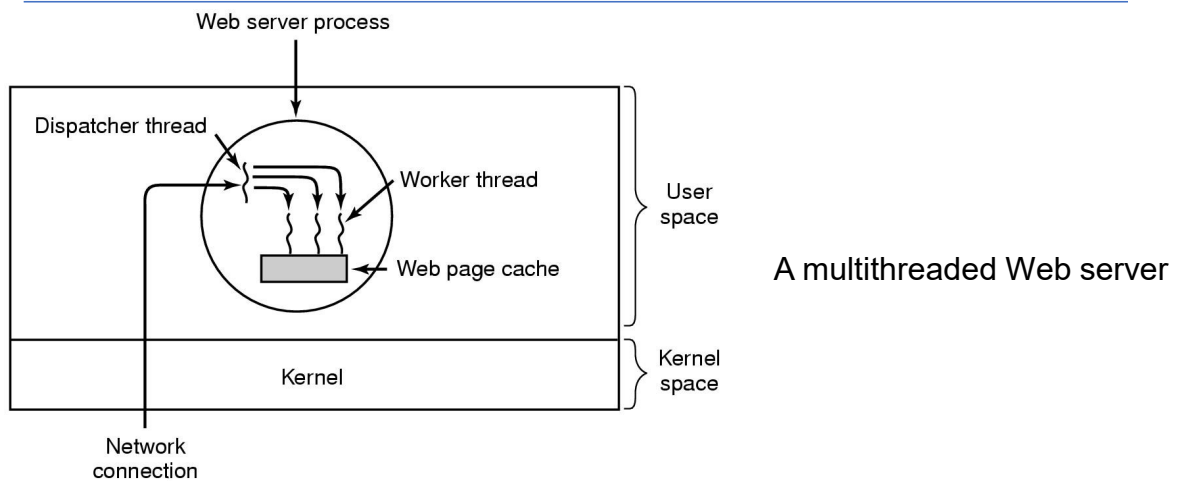
Why Threads?

- Perform operations in parallel on the same data
- Avoid complex explicit scheduling by applications
- Often more efficient than separate processes

Example: Web Browser

- Download page in one thread
- Actually, use a separate thread for each separately-downloaded image
- Let another thread respond to mouse clicks
- Maybe have a separate thread for each tab or page image

Web Browser – Thread Usage



Spring 2026

Operating Systems

55

55

Web Browser – Thread Usage

```
while (TRUE) {
    get_next_request(&buf);
    handoff_work(&buf);
}
```

(a)

```
while (TRUE) {
    wait_for_work(&buf);
    look_for_page_in_cache(&buf, &page);
    if (page_not_in_cache(&page))
        read_page_from_disk(&buf, &page);
    return_page(&page);
}
```

(b)

(a) Dispatcher thread
(b) Worker thread

Spring 2026

Operating Systems

56

56

Web Browser – Thread Usage

Model	Characteristics
Threads	Parallelism, blocking system calls
Single-threaded process	No parallelism, blocking system calls
Finite-state machine	Parallelism, nonblocking system calls, interrupts

Three ways to construct a server

More Examples

- Word processor
- Most graphical applications
- Anything where separate execution patterns need to share lots of data

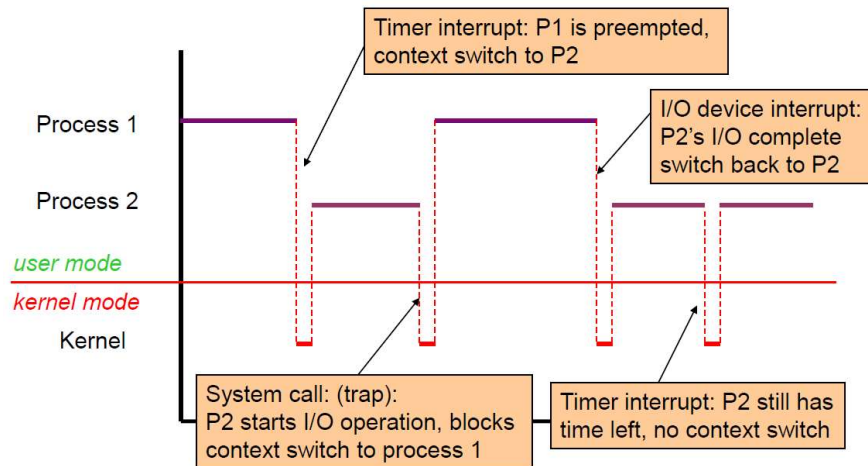
Context Switching

- Multiprogramming: switch to another process if current process is (momentarily) blocked
- Time-sharing: switch to another process periodically to make sure all process make equal progress
 - this switch is called a context switch

Context Switching

- Process of storing and restoring the state (context) of a process
 - execution can be resumed from the same point at a later time
 - enabling multiple processes to share a single CPU
- Context switches are usually computationally intensive and much of the design of operating systems is to optimize the use of context switches
- A context switch can mean a register context switch, a task context switch, a thread context switch, or a process context switch
 - determined by the processor and the operating system
 - switching requires a certain amount of time for saving and loading registers and memory maps, updating various tables and lists etc.

Context Switching



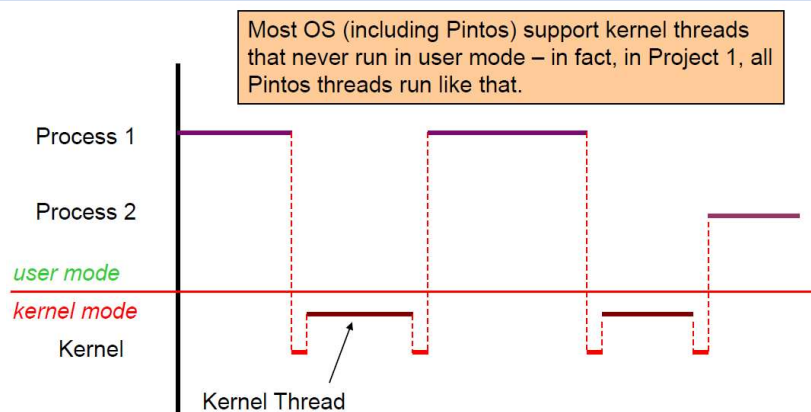
Spring 2026

Operating Systems

61

61

Aside: Kernel Threads



Careful: "kernel thread" not the same as kernel-level thread (KLT) – more on KLT later

Spring 2026

Operating Systems

62

62

Mode Switching

- A user process undergoes a mode switch when it needs access to system resources
 - Implemented through the system call interface or by interrupts such as page faults
- There are two modes:
 - User mode – Programs that execute in the user protection domain are user processes
 - Kernel mode – Programs that execute in the kernel protection domain include interrupt handlers, kernel processes, the base kernel, and kernel extensions (device driver, system calls and file systems)
- Mode switches
 - The use of a system call by a user-mode process allows a kernel function to be called from user mode.
 - Access to functions that directly or indirectly invoke system calls is typically provided by programming libraries, which provide access to operating system functions

Mode Switching

- User → Kernel mode
 - For reasons external or internal to CPU
- External (hardware) interrupt:
 - timer/clock chip, I/O device, network card, keyboard, mouse
 - asynchronous (with respect to the executing program)
- Internal interrupt (software interrupt, trap, or exception)
 - are synchronous
 - can be intended: for system call (process wants to enter kernel to obtain services)
 - or unintended (usually): fault/exception (division by zero, attempt to execute privileged instruction in user mode)
- Kernel → User mode switch on **iret** instruction

Mode Switching

- When a transition between user mode and kernel mode is required in an operating system, a **context switch** is not necessary
- Mode transition is not by itself a context switch
- However, depending on the operating system, a context switch may also take place at this time

Context vs Mode Switching

- Mode switch guarantees kernel gains control when needed
 - To react to external events
 - To handle error situations
 - Entry into kernel is controlled
- Not all mode switches lead to context switches
 - Kernel code's logic decides when – subject of scheduling
- Mode switch always hardware supported
 - Context switch (typically) not – this means many options for implementing it!

Thread Usage

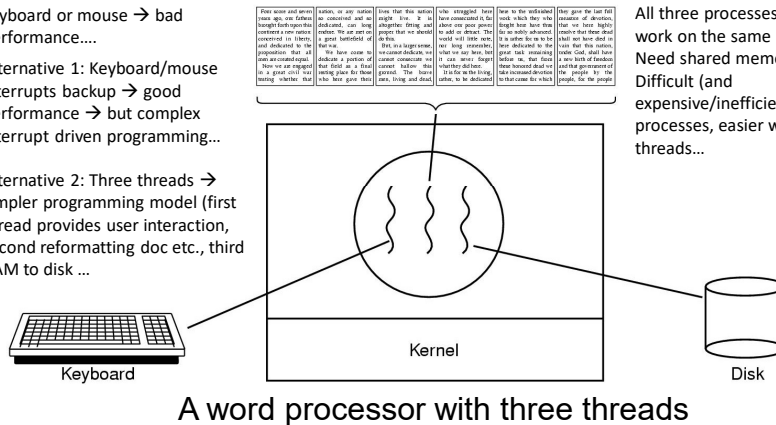
If single threaded: a disk backup will block commands from keyboard or mouse → bad performance....

Alternative 1: Keyboard/mouse interrupts backup → good performance → but complex interrupt driven programming...

Alternative 2: Three threads → simpler programming model (first thread provides user interaction, second reformatting doc etc., third RAM to disk ...)

Why not three processes?

All three processes need to work on the same document. Need shared memory... Difficult (and expensive/inefficient) with processes, easier with threads...

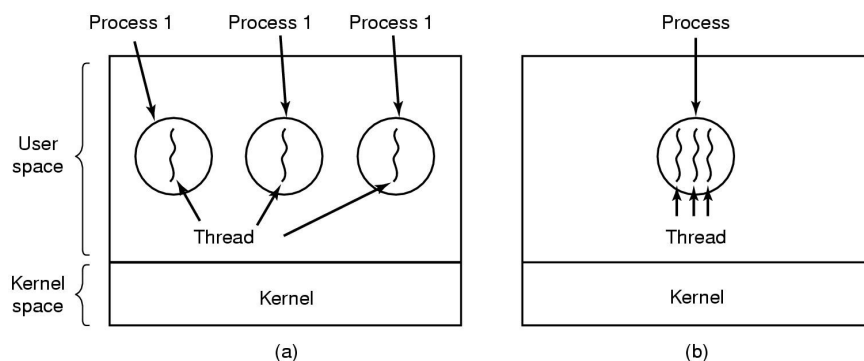


Spring 2026

Operating Systems

67

The Classical Thread Model (1)



Spring 2026

Operating Systems

68

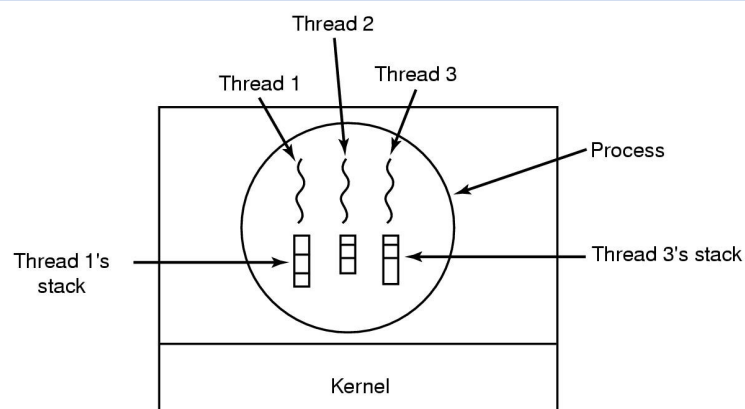
The Classical Thread Model (2)

Per process items	Per thread items
Address space	Program counter
Global variables	Registers
Open files	Stack
Child processes	State
Pending alarms	
Signals and signal handlers	
Accounting information	

The first column lists some items shared by all threads in a process

The second one lists some items private to each thread

The Classical Thread Model (3)



Each thread has its own stack

POSIX Threads (1)

Thread call	Description
Pthread_create	Create a new thread
Pthread_exit	Terminate the calling thread
Pthread_join	Wait for a specific thread to exit
Pthread_yield	Release the CPU to let another thread run
Pthread_attr_init	Create and initialize a thread's attribute structure
Pthread_attr_destroy	Remove a thread's attribute structure

Some of the Pthreads function calls

POSIX Threads (2)

```
#include <pthread.h>
#include <stdio.h>
#include <stdlib.h>

#define NUMBER_OF_THREADS 10

void *print_hello_world(void *tid)
{
    /* This function prints the thread's identifier and then exits. */
    printf("Hello World. Greetings from thread %d\n", tid);
    pthread_exit(NULL);
}

int main(int argc, char *argv[])
{
    /* The main program creates 10 threads and then exits. */
    pthread_t threads[NUMBER_OF_THREADS];
    int status, i;

    for(i=0; i < NUMBER_OF_THREADS; i++) {
        printf("Main here. Creating thread %d\n", i);
        status = pthread_create(&threads[i], NULL, print_hello_world, (void *)i);

        if (status != 0) {
            printf("Oops. pthread_create returned error code %d\n", status);
            exit(-1);
        }
    }
    exit(NULL);
}
```

An example program using threads

Per-thread Stack

- Each thread has its own stack
 - What: Containing one frame for each procedure called but not yet returned from
 - Why: Each thread generally will call different procedures → different execution history

Implementing Threads

- Three basic strategies
- User-level: possible on most operating systems, even ancient ones
- Kernel-level: looks almost like a process (i.e., Linux, Solaris)
- Scheduler activations (Digital Tru Unix 64, NetBSD, some Mach)

User-Level Threads

- Thread creation, destruction, and scheduling done at user level
- To create a thread, must allocate storage for stack, program counter, registers, state, etc, in a thread table
- When a thread yields the CPU or blocks, the thread library saves everything in the thread table
- The thread scheduler finds the highest-priority thread that's ready to run
- Its registers, program counter, etc., are restored from its thread table

User-Level Threads

- Put thread package entirely in user space
- Kernel thinks/concerns running single-thread processes
- User level threads can be implemented on OSs not supporting threads
- Implemented by a library
- Thread tables – analogous to kernel's process table
 - Each process needs one for its threads
 - Program counter, stack pointer, registers, state, ...
 - Managed by the run-time system
 - Faster context switching

Efficiency Gains

- No system calls needed
- No context switch
- No copying data safely across a protection boundary
- No need to flush the hardware memory cache

Other Advantages

- Allows each process to have its own customized scheduling algorithm
 - E.g., garbage collector thread
- Scales better than kernel-level threads
 - Kernel-level threads may require table & stack space in the kernel-level → problematic if when large number of threads

Disadvantages of User-Level Threads

- Blocking system calls
- Page faults
- Inability to benefit from multiprocessor CPUs
- Kernel can't automatically grow per-thread stacks

Growing the Stack

- The system allocates a certain amount of memory for the initial piece of the stack
- References to other potential parts of the stack cause a trap
- If this trap looks like an attempt to use an unallocated part of the stack, the kernel allocates more memory and extends the stack
- This only works if the kernel knows where the stack is

Blocking System Calls

- What if a thread issues, say, a **read()** call and there's no data available?
- The kernel will make the whole process block because it's unaware of the thread structure
- This is contrary to the purpose of having threads
 - Difficult issue in user level threads → need modifications to OS, e.g. making all system calls to be nonblocking...
 - Another alternative in two slides...

Page Faults

- Part of the virtual memory system; won't say much now
- Briefly, though — a trap occurs because referenced memory isn't available
- No ability to suspend just one thread

Wrapper Functions

- The thread library can contain alternate versions of system call functions
- These versions check for blocking before calling the kernel
- Example: do a non-blocking select() or poll() before a read()
- If the call would block, fire up another thread instead
- If no threads are ready, do a blocking select() or poll() and let the kernel run a different process

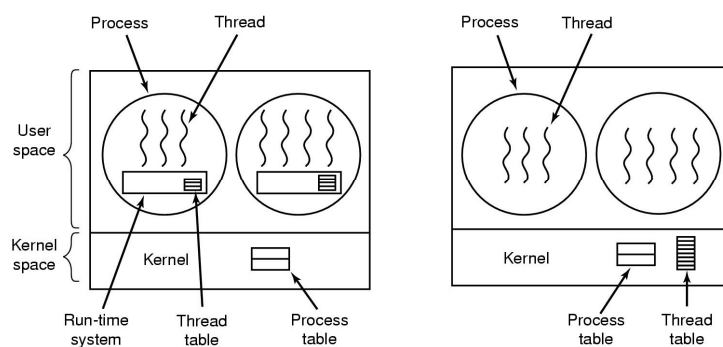
Spring 2026

Operating Systems

83

83

Implementing Threads in User Space



(a) A user-level threads package

(b) A threads package managed by the kernel

Spring 2026

Operating Systems

84

84

Kernel-level Threads

- Kernel manages the thread
 - To create/destroy a thread, the originating thread makes a kernel call
- Similar thread table, but in the kernel
 - Often integrated with process table
- No problem with blocking system calls — ordinary process scheduling does the right thing
- Handles multi-CPU case easily
- But — thread-switching is expensive, since it needs a context switch
- Still cheaper than process switches, since you can use the same virtual memory map

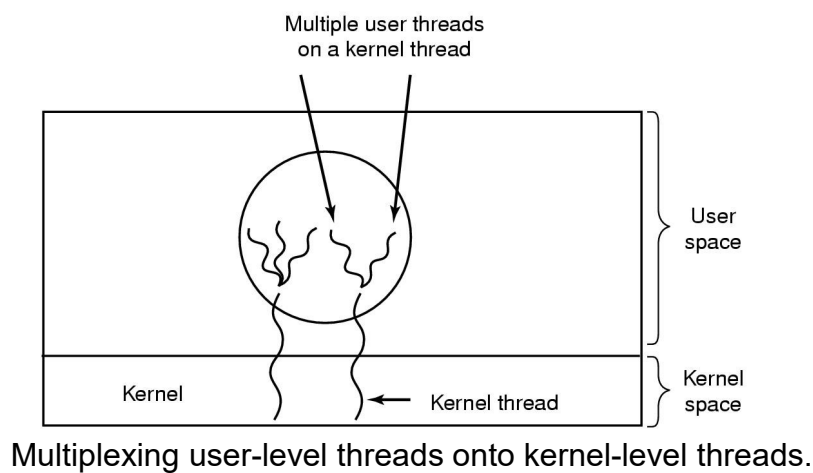
Kernel-level Threads – Issues

- Fork
 - What happens to the threads – does the new process have as many threads?
 - Depends on what follows the fork...
- How about signals
 - They are sent to processes, not threads (in many models)
 - Solution: let the thread register to receive the signal – when signal comes, it is passed to it...
 - What if more than one thread register for the same signal

Other Schemes

- Hybrid: some kernel-level threads, with each kernel-level thread supporting several user-level threads
 - User-level threads are multiplexed to the kernel-level threads
 - Kernel is aware of only the kernel-level threads and schedules those
- Scheduler activations
- Pop-up threads

Hybrid Implementations



Scheduler Activations

- Get the best of both worlds — the efficiency of user-level threads and the non-blocking ability of kernel-level threads
- Relies on upcalls

What's an Upcall?

- Normally, user programs call functions in the kernel
- Sometimes, though, the kernel calls a user-level process to report an event
- Sometimes considered unclean — violates usual layering

Scheduler Activations

- Thread creation and scheduling is done at user level
- When a system call from a thread blocks, the kernel does an upcall to the thread manager
- The thread manager marks that thread as blocked, and starts running another thread
- When a kernel interrupt occurs that's relevant to the thread, another upcall is done to unblock it

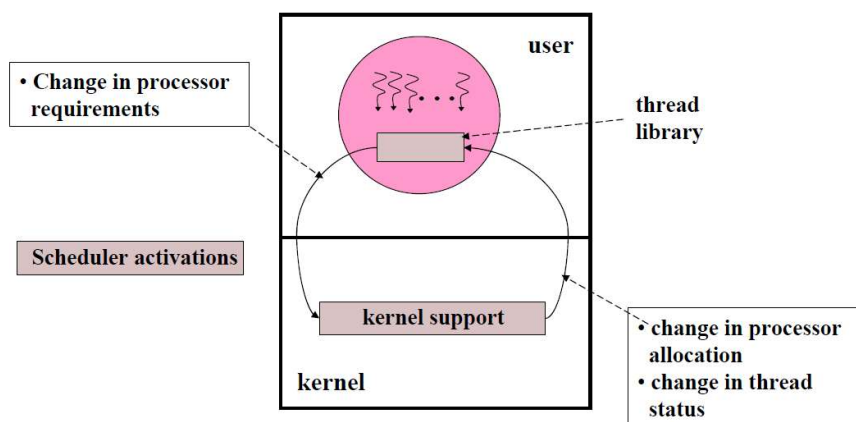
Spring 2026

Operating Systems

91

91

Scheduler Activations



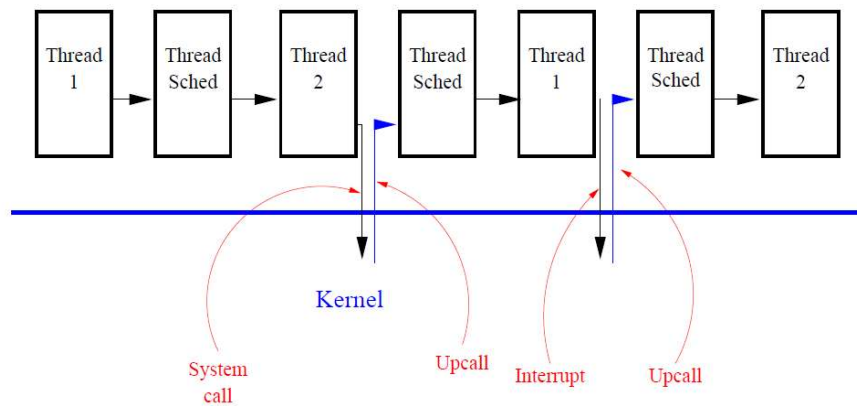
Spring 2026

Operating Systems

92

92

Scheduler Activations



Spring 2026

Operating Systems

93

93

Pop-Up Threads

- When a message arrives, the kernel creates a new thread
- Sometimes, the thread can run entirely in the kernel
- Messy and delicate — what process should own the thread, what sorts of events result in thread creation, etc.

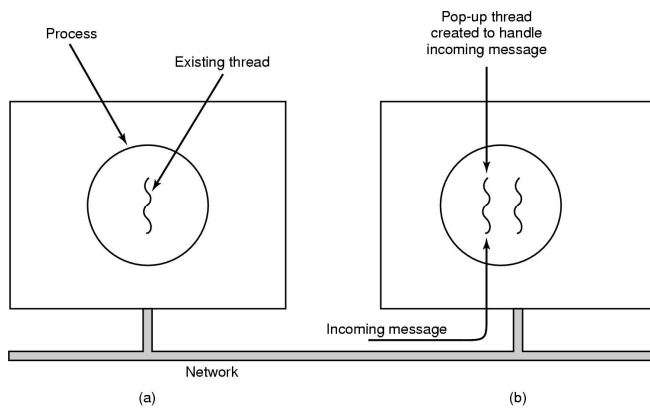
Spring 2026

Operating Systems

94

94

Pop-Up Threads



Creation of a new thread when a message arrives.

(a) Before the message arrives.

(b) After the message arrives.

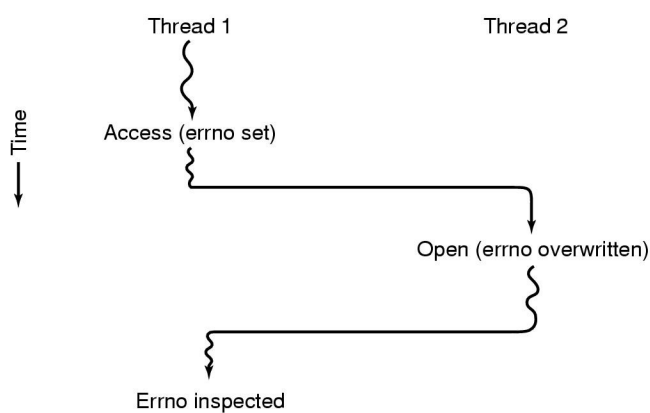
Spring 2026

Operating Systems

95

95

Making Single-Threaded Code Multithreaded (1)



Conflicts between threads over the use of a global variable.

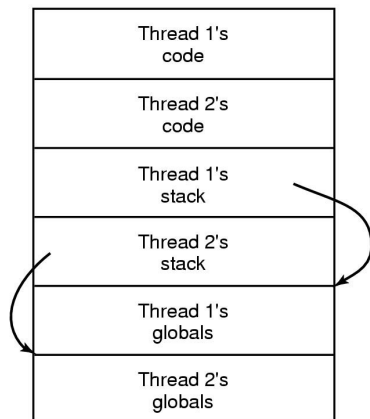
Spring 2026

Operating Systems

96

96

Making Single-Threaded Code Multithreaded (2)



Threads can have private global variables

Using Threads

- Library routines may be used by more than one thread simultaneously
- Not all routines are prepared for this
- Static variables can be overwritten by another thread
- Must use reentrant routines

What is a Reentrant Routine?

- A routine that can be invoked more than once simultaneously
- Can only use local variables or dynamically allocated variables
- Must not use static variables
- Many C library routines do use static buffers!

Normal and Reentrant Versions

Normal `struct tm *localtime(const time_t *timep);`

Reentrant `struct tm *localtime_r(const time_t *timep, struct tm *result);`

The normal version stores the result in a static buffer and passes a pointer back; the reentrant version requires the caller to supply a buffer.

What if You Need Global Variables?

- Sometimes, there are global variables you have to use
- The whole purpose of threads is to share access to certain data
- Must synchronize access, but this will come later...

Signals

- Short messages to a process
- More accurately, an interrupt sent to a process, much like hardware interrupts to the OS
- Often, signals not explicitly fielded will terminate the process

Types of Signals

- **Environment**
 - Hangup, ^C, coredump, power failure
- **Bugs**
 - Assertion failed, memory error, floating point error, resource exceeded, pipe closed, bad system call, timeout
- **Debugging**
 - Tracing, profiling
- **I/O**
 - I/O possible, urgent message, window size change, background process wants to talk to terminal
- **Other**
 - Many more

Default Signal Actions

- Ignore — harmless
- Terminate program
- Create a core dump and terminate program
- Suspend program
- Resume program

Programs and Signals

- Send a signal to a process or thread
- Call a subroutine when a signal occurs
- Temporarily block signals
- Test for or wait for a signal

Sending a Signal

- Signals can be generated by the kernel
- Example: when the terminal handler sees ^C, it sends SIGINT to all processes associated with that terminal
- Signals can be sent explicitly by another process: `kill(pid, signum)`
- The shell's `kill` command translates directly into the `kill()` system call
- The default signal is catchable; you have to be more forceful to deal with that if you really want to terminate the process. . .

Catching Signals

- The `sigaction()` subroutine specifies what to do when a signal occurs
- Choices: default action, call a subroutine, ignore the signal
- `SIGKILL` can't be caught; it always terminates the program
- When a signal is caught, other signals can be blocked until the subroutine returns

Blocking Signals

- `sigprocmask()` blocks some signals
- The signals remain pending; when unblocked, they will occur
- `SIGKILL` cannot be blocked, either

Suspending a Process

- `sigsuspend()` suspends a process and changes the set of blocked signals
- Cannot accomplish the same thing with
`sigprocmask(...);`
`sleep(...);`
- There's a race condition — what if the signal occurs between the two statements?

Race Conditions

<code>sigprocmask(...);</code>	<code>sigprocmask(...);</code>
Signal occurs	<code>sleep(...);</code>
<code>sleep(...);</code>	Signal occurs
Never wakes up	Process awakened by signal

Correct behavior here depends on which event occurs first. Testing is unlikely to find the flaw, since it's possible but improbable. To avoid race condition problems, you need to rely on atomic operations, such as `sigsuspend()`

Signals and Processes

- Signal state is inherited by child processes
- Signal state is partially preserved across `exec()` calls:
 - Ignored signals remain ignored
 - Caught signals are reset to default action
- Forgetting about ignored signals can cause problems for child processes

Signals and Threads

- Signal handlers are shared among all threads in a process
- Each thread can have its own signal mask
- If a signal occurs, it's randomly sent to a single thread that doesn't block it

Signals and Interrupts

- Unix signals are the closest thing to hardware interrupts that you see in application programs
- Dealing with them can be just as messy. . .
- Issues include race conditions, blocking signals, reentrancy, and more

See you next time!